# GPU-Assisted AES Acceleration for Bulk Data Processing in Telecommunications: Experimental Evidence and Practical Constraints

**Marko Mihajlovic**[1]

**Abstract:**

High-throughput cryptographic processing is an important requirement in modern telecommunications infrastructure, where GPU acceleration offers opportunities for improving bulk data encryption. This study investigates the conditions under which GPU-assisted AES-128 processing can provide measurable benefits over hardware-accelerated CPU encryption. A validation-first experimental approach evaluates AES in CTR and ECB modes across three heterogeneous computing configurations, using paired workloads, five measurement sessions per system, and separate device-resident and synchronous transfer-inclusive GPU execution boundaries. For 32 MiB AES-CTR workloads, the GPU processing pipeline achieves median paired speedups of 1.354× on the T600 Laptop GPU and 4.761× on the Tesla T4 relative to their respective AES-NI CPU implementations, with improvements observed in all five sessions. In contrast, the GTX configuration achieves only 0.935×, demonstrating that acceleration is not universally beneficial. The results establish experimentally supported conditions under which GPU processing can improve bulk AES throughput despite synchronous data transfers. These findings provide experimental evidence that GPU-assisted AES-CTR processing can improve bulk encryption throughput over hardware-accelerated CPU execution, even when synchronous data transfers are included. The demonstrated gains identify concrete opportunities for accelerating cryptographic workloads in heterogeneous telecommunications systems, although improvements in complete IPsec and authenticated network processing remain to be established.

**Keywords:** Advanced Encryption Standard (AES), AES-NI, GPU acceleration, CUDA, heterogeneous computing, cryptographic benchmarking, data-transfer overhead, reproducible performance evaluation.

## 1 Introduction

Telecommunications infrastructure must protect increasing volumes of data while maintaining throughput and acceptable processing latency. Communication gateways, network services, and distributed processing systems therefore require efficient cryptographic operations. The Advanced Encryption Standard (AES), specified in NIST FIPS 197 [1], is a fundamental building block of symmetric

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

encryption. Its practical performance depends on hardware acceleration, execution strategy, and data movement between processing resources, as well as the cryptographic algorithm.

Hardware-assisted CPU encryption provides the reference against which an alternative processing path must be evaluated. Graphics processing units (GPUs) offer parallel execution that can benefit workloads containing many independent AES operations. Counter (CTR) mode permits independent counter blocks to be encrypted in parallel to generate keystream [2], making it suitable for investigating GPU-assisted bulk encryption. Large buffers and collections of independent data units are potential workloads for telecommunications-oriented systems that already contain general-purpose GPUs. Such systems could use existing heterogeneous computing resources for bulk encryption without requiring a dedicated cryptographic accelerator. Whether this is beneficial depends on the costs of moving the workload through the GPU as well as the throughput of the encryption operation itself.

Using these resources for encryption introduces host-to-device transfers, kernel execution, synchronization, and transfers back to host memory. These costs can reduce or eliminate the computational advantage of parallel execution. The relevant question is therefore whether GPU-assisted AES processing provides a reproducible bulk-throughput improvement over hardware-accelerated CPU execution, and which workload, platform, and data-movement conditions determine that improvement. The study distinguishes performance with data already resident in GPU memory from performance that includes synchronous host–device transfers and synchronization.

This distinction also affects the interpretation of potential processing strategies. Encrypting a contiguous buffer, aggregating independent requests, and operating on device-resident data involve different data-placement and execution costs. Batching may provide more parallel work per invocation, but its usefulness depends on workload size and application latency requirements. A favorable result for one execution boundary does not establish an advantage for the others; each must be assessed against the available host implementation.

The study evaluates validated AES-128 implementations in Electronic Codebook (ECB) and CTR modes across three heterogeneous computing configurations. CTR is the primary workload of interest, while ECB provides a controlled reference for independent block processing. Hardware-assisted CPU AES-NI serves as the baseline. Cryptographic correctness verification, controlled inputs, paired execution conditions, repeated measurement sessions, and explicit timing boundaries support the comparison. Section 3 specifies the implementations, workload sizes, and session-level analysis.

The contribution is experimental evidence identifying conditions under which GPU-assisted bulk AES processing improves throughput, including cases where an advantage remains after synchronous data movement. The measurements also identify configurations in which transfer costs or host implementation characteristics prevent consistent improvement. The common validation and measurement protocol provides a reproducible basis for assessing these outcomes across the tested systems. The study evaluates existing encryption approaches rather than introducing a new AES algorithm. Both favorable and unfavorable acceleration outcomes are used to delimit the operating conditions supported by the measurements. The results characterize the evaluated CPU and GPU paths; they do not establish a universal hardware advantage or the maximum throughput attainable by either architecture.

For telecommunications, bulk data protection, workload aggregation, and device-resident processing are potential applications of these findings. Their application-level effectiveness remains untested. In particular, improved AES-CTR buffer processing does not demonstrate acceleration of complete network security protocols. Authenticated encryption additionally requires authentication processing, while practical protocols introduce packet handling, cryptographic state management, and data-delivery costs. These operations, along with batching and queueing latency, require separate evaluation before the measured bulk-processing improvements can be extended to telecommunications applications.

Section 2 reviews GPU AES implementations, CPU–GPU comparison methods, and the evidence needed to assess offloading. Section 3 describes the experimental design, validation, timing boundaries, and statistical analysis. Section 4 presents the measured acceleration outcomes, and Section 5 examines their practical constraints and potential telecommunications applications. Section 6 summarizes the findings.

## 2 Related Work and Research Gap

Early contributions by Manavski [3] and Harrison and Waldron [4] established the feasibility of AES execution on programmable GPU architectures. Subsequent work investigated how parallel processing, memory organization, and architecture-specific optimization affect encryption throughput. The available parallelism depends partly on the encryption mode: ECB permits independent block transformations, while CTR generates keystream from distinct counter values [2]. Both expose independent work for GPU threads when buffers are sufficiently large. ECB is unsuitable for protecting general structured data and serves here only as a controlled performance reference.

Iwai et al. [5] examined thread granularity, memory organization, and the interaction between computation and data transfers, including overlapping transfers

with computation. Their CUDA implementation achieved approximately 35 Gbit/s for device-side AES processing and approximately 22.5 Gbit/s for transfer-aware execution. These results demonstrate that the execution boundary and transfer strategy affect the reported encryption rate. Wang and Chu [6] investigated lookup-table-based AES implementations and GPU execution strategies. Tezcan [7] examined shared-memory optimization and bank-conflict avoidance, reporting AES-128 encryption throughput of 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

More recent work has explored bit-sliced cipher representations. A 2024 study [8] reported high-performance AES-CTR and AES-ECB execution on desktop and embedded NVIDIA platforms. Together, these studies demonstrate the computational capabilities of optimized GPU implementations. Their reported rates must nevertheless be interpreted within their respective measurement boundaries: device-side encryption throughput does not directly specify the rate at which an application can receive, encrypt, and deliver data. It also does not establish whether offloading improves performance relative to the host processor available in a particular system.

That comparison depends on the CPU implementation. Dedicated instructions such as AES-NI reduce the cost of AES transformations [9] and provide a hardware-assisted baseline for evaluating GPU offloading. Lee et al. [10] demonstrated that conclusions about heterogeneous acceleration depend strongly on the optimization level of the implementations being evaluated. Their findings support using an appropriately accelerated CPU reference when deciding whether GPU offloading offers an additional processing benefit. Accordingly, comparing GPU encryption with conventional unaccelerated software cannot establish an advantage over the host's hardware-assisted AES path. The reference must represent the execution path against which an offloading decision will actually be made, and the comparison remains specific to the implementations evaluated.

Data location further determines which costs enter that decision. Gregg and Hazelwood [11] showed that CPU–GPU performance evaluation requires explicit consideration of data movement. An advantage for device-resident input may shrink when both input and output cross the host–device boundary. CUDA guidance recommends minimizing unnecessary transfers, using appropriate synchronization, and considering overlap between communication and computation [12, 13]. The transfer-aware AES work of Iwai et al. [5] demonstrates that this concern is already established in GPU cryptographic processing.

For communication infrastructure, these findings connect encryption performance to the surrounding processing path. A host-managed interface may receive data, organize buffers, invoke encryption, and forward the output. The resulting costs differ between contiguous buffers, aggregated independent messages, and individual latency-sensitive packets. Device-resident measurements and

transfer-inclusive measurements therefore answer different questions. The present study distinguishes resident execution from a synchronous prepared-buffer pipeline containing host-to-device and device-to-host transfers. This pipeline measures a broader operation than the kernel alone, but excludes complete network input/output and application processing; it also does not evaluate transfer–computation overlap.

Telecommunications security adds requirements beyond encryption throughput. AES-GCM combines counter-mode encryption with GHASH authentication [14]; an improvement in AES-CTR can therefore concern one component of an authenticated-encryption pipeline without establishing an equivalent improvement in IPsec or TLS processing. Practical implementations must also account for nonce management, packet preparation, synchronization, and output delivery. These requirements distinguish a bulk encryption benchmark from a complete security service, even when the underlying cipher operations are parallelizable.

Workload aggregation introduces a further throughput–latency trade-off. Combining small messages may increase the work available to the GPU, while waiting for a batch adds queueing delay. Bulk data protection, traffic aggregation, heterogeneous communication gateways, and data-processing services are consequently possible application contexts, but their suitability depends on host acceleration, data placement, and operational requirements. Peak kernel rates alone cannot determine that suitability. The present measurements characterize contiguous bulk buffers and do not demonstrate packet batching or complete telecommunications protocol performance.

Assessing whether an observed advantage is repeatable also requires control of experimental variability. Mytkowicz et al. [15] showed that experimental conditions can introduce systematic bias and alter performance results. Kalibera and Jones [16] emphasized the importance of accounting for variability across multiple levels of experimental replication, while Hoefler and Belli [17] discussed scientifically defensible performance evaluation of parallel computing systems. These contributions provide a methodological basis for determining whether an acceleration is repeatable under the conditions tested. These studies motivate reporting repeated observations and their execution context rather than relying on the highest throughput measurement. For cryptographic workloads, functional correctness must additionally be verified before timings are accepted as evidence of encryption performance.

The present evaluation applies these principles through cryptographic validation, paired workloads, repeated study sessions, documented timing boundaries, and preservation of raw observations and experimental provenance. Session-level evidence distinguishes isolated favorable measurements from advantages observed consistently across executions, while retaining outcomes that favor the CPU. Detailed procedures and aggregation equations are given in Section 3.

The literature thus establishes GPU AES feasibility, optimization techniques, and the need to account for transfers, CPU acceleration, and measurement variability. The question addressed here is which operating conditions produce a repeatable bulk-processing benefit when the host already supports hardware-assisted encryption. High kernel throughput alone cannot answer this question, and an unfavorable transfer-inclusive result on one system cannot settle it for other platforms or workloads.

To examine these conditions, the study compares AES-128 ECB and CTR workloads of different sizes across three heterogeneous configurations under a common validation and measurement protocol. The comparison uses hardware-assisted CPU AES-NI as its reference and retains session-level measurements for assessing consistency across repeated executions. Its contribution is the identification of experimentally supported operating conditions in which GPU-assisted bulk AES processing provides an advantage, together with evidence of constraints under which that advantage disappears or becomes inconsistent. This supports assessment of GPU-assisted bulk encryption and workload placement within the tested execution boundaries. Extending those findings to authenticated encryption and network processing requires separate application-level measurements.

## 3 Methodology

The evaluation compares AES-128 encryption using hardware-assisted CPU AES-NI and a validated CUDA implementation. CTR is the primary workload; ECB provides a controlled reference for mode-dependent implementation behavior. Both modes process contiguous buffers under the same validation and measurement protocol. Three complete hardware–software configurations are evaluated: two Windows laptop systems and a Linux-based cloud allocation. Table 1 lists the recorded hardware and software. The exact model of the cloud host CPU was unavailable in the archived environment records.

The CPU reference uses PyCryptodome with hardware-assisted AES-NI explicitly enabled [9, 18]. Native backend initialization was checked for both modes; executed processor instructions were not traced directly. The GPU path uses the validated `cuda_baseline` kernel through CuPy's custom kernel interface [19]. Each CUDA thread processes one 16-byte AES block, with 128 threads per CUDA block. These paths are selected from a broader campaign that evaluated additional implementations. The analysis retains their original execution context, including interleaved jobs excluded from the selected results.

Cryptographic validation precedes acceptance of timing measurements. Known-answer tests and consistency checks against CPU reference operations use the AES specification and mode definitions [1, 2]. The archived records

Table 1
Hardware and software configurations used for AES acceleration experiments.

| ID | Hardware and environment | Software stack |
|---|---|---|
| GTX | Intel Core i5-10300H; GeForce GTX 1650 Ti, 4 GB; Windows 10 | Driver 610.62; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0 |
| T600 | Intel Core i7-11850H; T600 Laptop GPU, 4 GB; Windows 11 | Driver 581.95; CUDA runtime 12.9; CuPy 14.2.0; PyCryptodome 3.24.1 |
| T4 | Intel Xeon at 2.00 GHz, two exposed logical processors; Tesla T4; Linux/Google Colab | Driver 580.82.07; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0 |

document successful CUDA execution and agreement with expected outputs. Executions failing the required checks are excluded from valid performance results. Experimental configuration, measurement-record completeness, and source-code identity are also verified.

The archived harness validates each measured output against the CPU reference outside the timed call, in addition to the initial validation gate. An output mismatch stops performance export. For each size, mode, and repetition, a single synthetic message is replayed across the paired backends. Plaintext and a 16-byte key are generated from the seeded random generator. The initial CTR block consists of eight generated bytes followed by eight zero bytes. The CPU CTR call uses an empty nonce and interprets the full initial counter as a big-endian integer.

Pair identifiers record size, mode, and repetition. A SHA-256 digest of the concatenated key, counter, and plaintext records the common input material. Raw records also retain the session identifier, seed, backend, mode, measurement scope, job order, global execution order, and validation flag. These fields allow the selected CPU and GPU observations to be matched without pooling observations from different sessions. Previous output buffers are released outside the next timed call.

The retained dataset is checked against the archived file hashes. For each platform, mode, size, and repetition, the selected observations contain one CPU transformation, one resident GPU measurement, and one pipeline measurement with matching input digests. Each configuration group contains all twenty recorded repetitions. Throughput is independently checked against the corresponding byte count and recorded time before session-level aggregation. Resident records specify one kernel iteration, and the GPU records identify pinned host memory. The verification reconstructs the aggregates from raw records rather than relying solely on the saved summary tables.

The experiment uses five input sizes:

$$N \in \{1\ \text{KiB},\ 16\ \text{KiB},\ 1\ \text{MiB},\ 16\ \text{MiB},\ 32\ \text{MiB}\}. \tag{1}$$

Corresponding CPU and GPU executions use deterministic input generation and matching plaintext, encryption keys, and mode-specific parameters. Each platform has five study sessions with random seeds 2003–2007. For each measured configuration, five warm-up executions precede twenty recorded repetitions; warm-ups are excluded from the timing records. Each session contains thirty selected configuration groups, comprising two modes, five sizes, and three execution boundaries. With twenty observations per group, each session contributes 600 observations, giving

$$3 \times 5 \times 30 \times 20 = 9\,000 \tag{2}$$

raw timing observations across the three systems. Workload sizes execute in ascending order, while mode and backend/measurement-scope jobs are shuffled using the session seed. Benchmark jobs run sequentially. The selected observations were extracted from the existing campaign, without rerunning or reordering the benchmark schedule.

Three timing boundaries are retained:

1. **CPU AES-NI transformation.** The interval measures the host encryption call, including output allocation performed by that call. Cipher construction and key setup are excluded.

2. **GPU resident execution.** CUDA events measure a single kernel launch with input already in device memory. Host-to-device and device-to-host transfers are excluded.

3. **Prepared-buffer GPU pipeline.** The interval measures the synchronous host-driven sequence of host-to-device transfer, CUDA kernel execution, device-to-host transfer, and synchronization required to complete these operations.

CPU execution and the GPU pipeline are timed with Python's monotonic high-resolution `time.perf_counter_ns()` interface. Resident execution uses CUDA event timing with the required synchronization, following CUDA and CuPy measurement guidance [12, 20]. Thus, the CPU and pipeline intervals are measured from the host, while the resident interval measures device-side execution.

The prepared-buffer pipeline uses pinned host memory and preallocated device buffers, prepared before the timed interval. Its processing stages are represented as

$$T_{\text{pipeline}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}}, \tag{3}$$

where H2D and D2H denote host-to-device and device-to-host transfers, and the final term covers additional costs within the measured boundary. This expression describes the stages of the synchronous operation; it is not a decomposition calculated by adding separately aggregated component measurements. Recorded transfer-component intervals include synchronization effects and are not isolated measurements of physical PCIe transmission time.

GPU compilation, memory allocation and pinning, initial host-side input staging, key preparation, and application-level input/output are excluded from the pipeline interval. Transfers and computation are not overlapped using multiple asynchronous CUDA streams. Resident execution and prepared-buffer processing are measured separately, with the latter including the movement needed to process a host-resident buffer and return its output. No network input/output, packet aggregation, or authenticated-encryption operation is included in these measurements.

Elapsed processing time, throughput, and host-relative GPU acceleration are summarized at session level. For an input of $N$ bytes and elapsed time $T_{s,r}$ in seconds at repetition $r$ of session $s$, throughput in decimal MB/s is

$$R_{s,r} = \frac{N}{10^6 T_{s,r}}, \tag{4}$$

Input sizes use binary KiB and MiB units. Time and throughput medians are calculated independently within each session, then aggregated across the five session estimates:

$$\widetilde{T} = \text{median}_s \left( \text{median}_r \, T_{s,r} \right), \qquad \widetilde{R} = \text{median}_s \left( \text{median}_r \, R_{s,r} \right). \tag{5}$$

Throughput is summarized from the recorded per-repetition throughput values rather than obtained by inverting the aggregated latency statistic. For the corresponding CPU and GPU executions, acceleration is calculated from the paired session-level timing medians:

$$S_s = \frac{\widetilde{T}_{\text{AES-NI},s}}{\widetilde{T}_{\text{GPU},s}}, \qquad \widetilde{S} = \text{median}_{s=1,\ldots,5}(S_s). \tag{6}$$

Here, $\widetilde{T}_{\text{AES-NI},s}$ and $\widetilde{T}_{\text{GPU},s}$ denote within-session median times for the same platform, mode, and input size. GPU resident and pipeline speedups are calculated

separately against the corresponding AES-NI time. Values of $S_s > 1$ indicate a faster GPU path, while values below one indicate a faster CPU reference. The reported $\widetilde{S}$ is the median of five paired ratios, not a ratio of independently aggregated CPU and GPU throughput values.

The analysis reports session ranges, interquartile ranges, sample coefficients of variation, and counts of sessions with $S_s > 1$. For five ordered session estimates, the interquartile range is the fourth minus the second value; the sample coefficient of variation is the sample standard deviation divided by the mean. An advantage in every session is recorded as $5/5$ directional consistency. These are descriptive statistics, not confidence intervals or formal significance tests. The twenty repetitions within a session are not treated as independent sessions, and measurements are not removed merely because they appear extreme.

The use of repeated sessions, paired workloads, and documented execution conditions follows recommendations on experimental bias and hierarchical variability in performance measurement [15–17]. Raw observations and their provenance are retained for verification of the session estimates and acceleration calculations.

## 4 Results

Figure 1 presents median paired GPU speedups across the five input sizes, separately for AES-128 ECB and CTR and for resident and prepared-buffer execution. At 32 MiB, resident execution exceeds the corresponding AES-NI reference on all three systems, in both modes and in every session. Pipeline results differ by platform and mode: CTR gains occur in all five sessions on T600 and T4, whereas the remaining platform–mode combinations have mixed or consistently slower pipeline results.

Table 2 provides an overview of the 32 MiB findings, with resident and pipeline medians, pipeline session ranges, and directional counts. Bold pipeline values identify improvements observed in all five sessions. The table retains both modes and all three systems, including cases with a median below parity and cases whose session range crosses parity.

For T600 CTR, the median pipeline speedup of 1.354× corresponds to approximately 26.14% lower latency, calculated from the median paired CPU-to-GPU timing ratio. Its session range remains above parity, from 1.129× to 1.391×. The paired speedup has an interquartile range of 0.030896 and a sample coefficient of variation of approximately 8.18%. These statistics describe the five session-level ratios used for this configuration.

T4 CTR has the largest median pipeline speedup at 32 MiB, 4.761×, with a five-session range of 4.691–5.021×. For both T600 and T4 CTR, every recorded session favors the pipeline even though the measured interval includes transfers

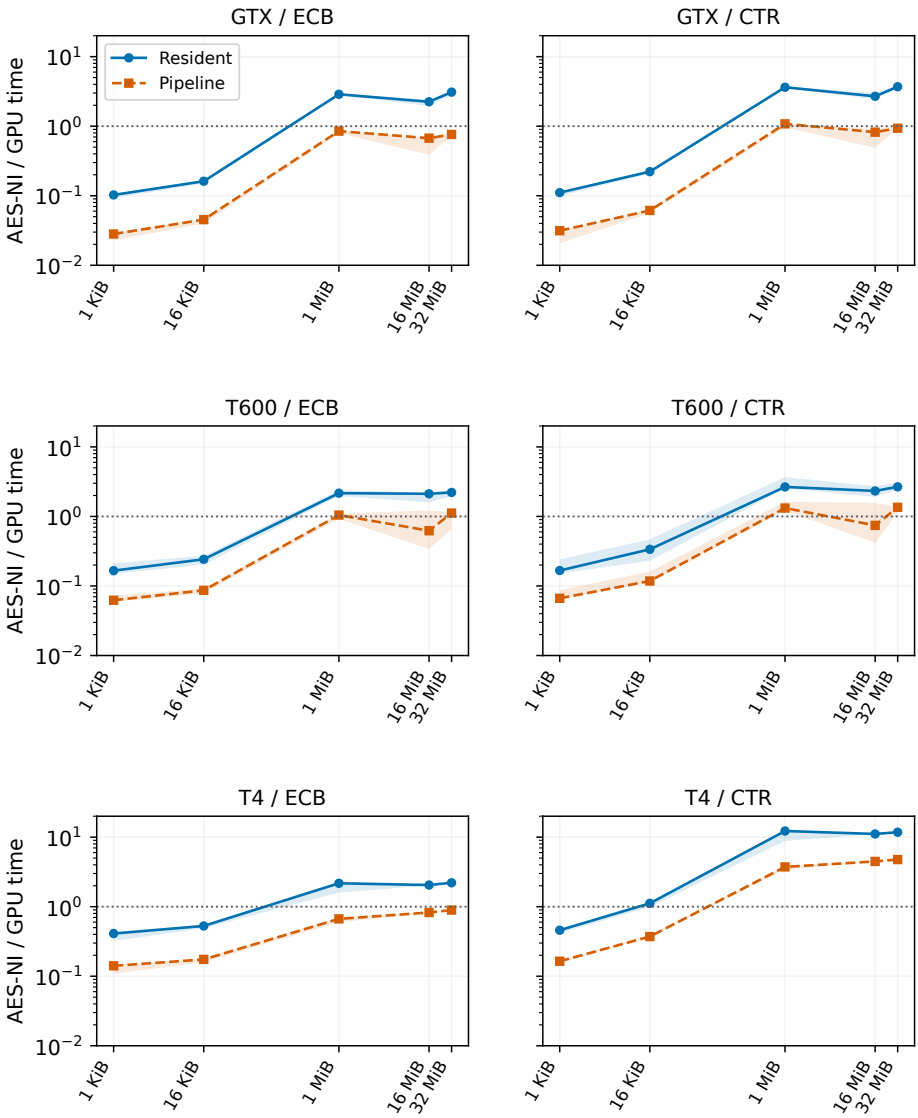



Fig. 1 — GPU-assisted AES-128 acceleration across input sizes and encryption modes. Curves represent median paired speedups relative to the corresponding host AES-NI implementation. Resident execution measures device-side computation, whereas the prepared-buffer pipeline includes synchronous host–device transfers and synchronization. Shaded regions represent the observed ranges across five study sessions, not confidence intervals. The horizontal reference denotes parity.

Table 2
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

| System | Mode | R | P | P range | P > 1 |
|---|---|---|---|---|---|
| GTX | ECB | 3.091 | 0.761 | [0.722, 0.818] | 0/5 |
| GTX | CTR | 3.693 | 0.935 | [0.918, 1.008] | 1/5 |
| T600 | ECB | 2.215 | 1.118 | [0.659, 1.144] | 3/5 |
| T600 | CTR | 2.666 | **1.354** | [1.129, 1.391] | **5/5** |
| T4 | ECB | 2.207 | 0.893 | [0.868, 0.901] | 0/5 |
| T4 | CTR | 11.758 | **4.761** | [4.691, 5.021] | **5/5** |

Bold pipeline medians and counts indicate acceleration in all five sessions. Resident execution is faster in all five sessions for every row.

and synchronization. The resident speedups are larger than the pipeline speedups on both systems, as they are for every platform–mode combination in Table 2.

GTX CTR has a pipeline median of 0.935× and favors the GPU in one session. Its range, 0.918–1.008×, crosses parity. GTX ECB remains below parity in all five sessions. T600 ECB has a median above one, 1.118×, but only three sessions favor the GPU, with a range of 0.659–1.144×. T4 ECB remains slower than AES-NI throughout the five sessions despite resident speedups above one. Thus, the positive T600 ECB median coexists with mixed session outcomes, while the T600 and T4 CTR medians accompany exclusively positive outcomes.

Table 3 reports absolute GPU throughput at the same input size. T4 has the highest resident and pipeline CTR rates among the tested systems. GTX resident CTR throughput exceeds T600 resident throughput, while the ordering is reversed for the pipeline. On each system, the pipeline rate is below the corresponding resident rate in both encryption modes.

Table 3
GPU throughput at 32 MiB in decimal MB/s. Values are medians of five session-level throughput medians.

| System | Mode | Resident | Pipeline |
|---|---|---|---|
| GTX | ECB | 4797.6 | 1207.8 |
| GTX | CTR | 4585.6 | 1188.9 |
| T600 | ECB | 4451.1 | 1997.8 |
| T600 | CTR | 4328.9 | 2049.4 |
| T4 | ECB | 9176.9 | 3713.0 |
| T4 | CTR | 9166.8 | 3713.9 |

The T4 GPU rates are nearly identical across modes: approximately 3713 MB/s

for ECB and 3714 MB/s for CTR in the pipeline, and 9177 and 9167 MB/s, respectively, in resident execution. In contrast, its host-relative pipeline speedups are 0.893× for ECB and 4.761× for CTR. The host AES-NI ECB/CTR throughput ratio has a session median of 5.378× and a range of 5.250–5.795×. The corresponding host ratios on GTX and T600 are approximately 1.26×. The T4 host therefore records higher ECB than CTR throughput in every session, while its measured GPU throughput changes little between modes. Figure 2 compares the absolute rates of all three execution paths, using the same vertical scale for both modes.

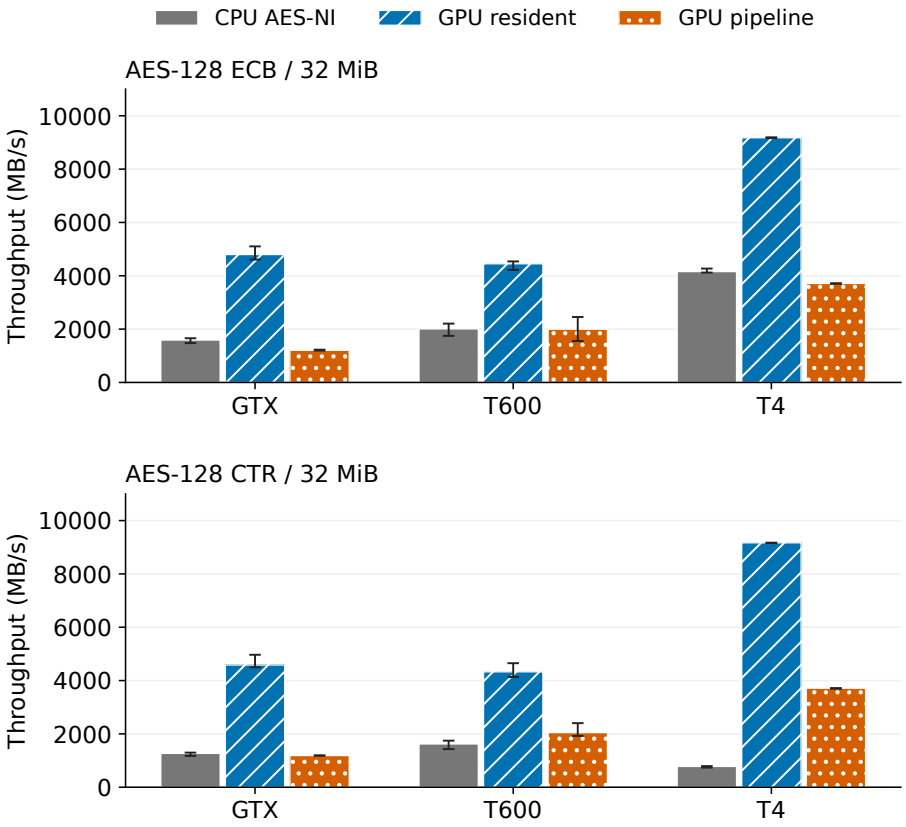



Fig. 2 — AES-128 throughput at 32 MiB for CPU AES-NI, resident GPU execution, and the prepared-buffer GPU pipeline. Bars show medians of five session-level throughput medians; error bars span the observed session minima and maxima, not confidence intervals. Throughput uses decimal MB/s. The GPU pipeline includes synchronous host–device transfers and synchronization; resident execution excludes these transfers. CPU bars represent the corresponding host implementation on each system.

The smaller workloads in Figure 1 show different acceleration outcomes. At

1 KiB, both resident and pipeline paths are slower than AES-NI in every session, on all systems and in both modes. At 16 KiB, all pipeline configurations remain slower in every session. Resident T4 CTR has a median above parity at that size, with gains in four sessions; the other resident configurations remain below parity throughout. At 1, 16, and 32 MiB, resident execution is faster than AES-NI in all sessions for every platform and mode.

Table 4 summarizes CTR pipeline results across the size grid. T600 gains occur in all sessions at 1 and 32 MiB, but in only one session at 16 MiB. T4 gains occur in all sessions at each of the three MiB-scale inputs. GTX has four favorable sessions at 1 MiB, none at 16 MiB, and one at 32 MiB; no tested size has five favorable GTX pipeline sessions. ECB pipeline results likewise contain no five-session improvement on GTX or T4. T600 ECB favors the GPU in four, two, and three sessions at 1, 16, and 32 MiB, respectively.

Table 4
CTR pipeline speedup across input sizes. Each entry gives the median paired speedup and, in parentheses, the number of sessions faster than AES-NI out of five. Bold entries have gains in all five sessions.

| Input | GTX | T600 | T4 |
|---|---|---|---|
| 1 KiB | 0.031 (0/5) | 0.066 (0/5) | 0.164 (0/5) |
| 16 KiB | 0.061 (0/5) | 0.118 (0/5) | 0.372 (0/5) |
| 1 MiB | 1.080 (4/5) | **1.320 (5/5)** | **3.734 (5/5)** |
| 16 MiB | 0.820 (0/5) | 0.744 (1/5) | **4.478 (5/5)** |
| 32 MiB | 0.935 (1/5) | **1.354 (5/5)** | **4.761 (5/5)** |

Absolute throughput is non-monotonic across the size grid. On GTX, throughput at 16 MiB is below both the 1 MiB and 32 MiB rates in all five sessions, for both modes and both GPU timing boundaries. On T4, this local reduction appears in resident throughput in four sessions for each mode, while neither pipeline mode shows the same pattern. The session counts therefore differ between platforms and between the two timing boundaries on T4.

## 5 Discussion

The experimental findings provide evidence that GPU-assisted AES processing can improve bulk encryption performance under identifiable operating conditions. Their significance for telecommunications lies not in a new AES algorithm, but in the possibility of using existing heterogeneous computing resources to accelerate selected cryptographic workloads.

The central observation is that the benefit of parallel AES execution can survive

host–device data movement. However, this improvement is conditional and depends on the encryption mode, processing volume, available CPU acceleration, and data location. These conditions must be considered when evaluating GPU-assisted encryption as a component of a larger communication system.

### 5.1 Implications for Bulk Cryptographic Processing

The observed AES-CTR improvements establish that GPU acceleration can be beneficial for sufficiently substantial workloads on particular hardware–software configurations. The positive prepared-buffer results are especially relevant because they demonstrate an advantage beyond isolated CUDA kernel execution.

For telecommunications systems, the main opportunity is the processing of substantial volumes of independent cryptographic work. Network gateways, communication servers, and data-processing services may encounter workloads in which multiple independent data units can be organized for parallel execution. AES-CTR is suitable for this form of processing because distinct counter blocks can be encrypted independently, provided that the counter and nonce requirements are satisfied [2]. GPU execution can exploit this independence by distributing AES transformations across many processing threads.

The measured improvement on the T600 configuration is relevant because it demonstrates that a repeatable bulk-processing advantage is not restricted to the tested datacenter-oriented configuration. Nevertheless, the results apply to the particular host CPU, GPU implementation, workload, and prepared-buffer execution boundary used in the experiment. The findings do not identify a universal minimum batch size. Although a 32 MiB workload produces a consistent advantage on selected systems, the experiments do not establish that this size is optimal or that larger workloads will necessarily improve performance further. More importantly, a contiguous encryption buffer is not equivalent to a group of independently managed network packets. Organizing packet data into a GPU workload requires additional processing whose performance was not evaluated.

Thus, the direct experimental contribution is the identification of favorable bulk-processing conditions. Their translation into packet-oriented workloads remains an architectural opportunity rather than a demonstrated deployment result.

### 5.2 Opportunities for Telecommunications Architectures

The findings suggest several ways in which GPU resources could complement existing CPU cryptographic acceleration in heterogeneous telecommunications systems.

One possibility is bulk processing within communication gateways or server-side data-protection services. Where a system already handles substantial con-

tiguous buffers, GPU-assisted AES may offer an additional processing path. The measured prepared-buffer results indicate that such an approach warrants consideration even when the host CPU supports AES-NI.

A second possibility is the aggregation of independent encryption requests into larger GPU workloads. This approach could increase the amount of parallel work available for each accelerator invocation and distribute execution overhead across multiple data units.

However, batching introduces a trade-off between throughput and latency. Waiting for sufficient work to form a batch can increase queueing delay, particularly during periods of low traffic. Packet metadata, independent cryptographic contexts, counter allocation, and result ordering also introduce requirements absent from the contiguous-buffer experiments. Consequently, the results support investigating batching for throughput-oriented workloads, but they do not establish an optimal batching policy or a guaranteed improvement in packet-processing performance.

A third opportunity concerns systems in which data are already processed on the GPU. If encryption can operate on device-resident buffers without additional host transfers, the relevant execution boundary differs from the synchronous host-to-host pipeline evaluated in this study.

The consistently favorable resident measurements provide evidence of the computational potential of this arrangement. Nevertheless, a practical device-resident application would still need to account for the acquisition, organization, authentication, and delivery of the protected data.

These opportunities point toward a workload-aware architecture in which CPU AES-NI remains available for smaller or latency-sensitive operations, while GPU processing is considered for suitable bulk workloads. Such an architecture is a design implication of the measurements. It was not implemented or experimentally evaluated as part of the present study.

### 5.3 Relationship to IPsec and Authenticated Encryption

AES-CTR is specified as a confidentiality mechanism for IPsec Encapsulating Security Payload (ESP) in RFC 3686 [21]. AES-GCM, which combines counter-mode encryption with authentication, is specified for IPsec ESP in RFC 4106 [22].

These standards establish a direct connection between parallelizable AES processing and standardized telecommunications security mechanisms. In particular, the independent generation of counter-mode keystream blocks provides opportunities for GPU-based parallel execution. However, the present experiments evaluate AES-128 CTR encryption rather than complete IPsec ESP or AES-GCM processing. The measured acceleration therefore characterizes the encryption component under the defined experimental conditions, not the performance of an

authenticated network-security protocol.

An IPsec implementation must account for operations that are not represented by the measured AES-CTR processing interval. These include packet framing, security-association state, nonce and sequence-number management, authentication, and the processing required to deliver protected packets. For AES-GCM, the additional GHASH computation and authentication-tag processing must also be considered. A measured improvement in the counter-mode encryption component does not determine the performance of the complete authenticated-encryption operation.

The findings are therefore relevant to evaluating the encryption component of possible GPU-assisted IPsec architectures, but they do not demonstrate that an IPsec tunnel or AES-GCM implementation would achieve the same acceleration factors.

A practical extension would need to preserve the security requirements of the selected protocol while measuring throughput and latency across the complete packet-processing path. In particular, independent packet contexts must not be combined in a way that compromises counter uniqueness, authentication boundaries, or the association between encrypted data and its protocol metadata.

### 5.4 Processing Constraints and Execution-Path Selection

The experiments identify several constraints that are relevant to the selection of a cryptographic execution path.

The first is data location. Device-resident execution avoids host–device transfers within the measured interval, whereas the prepared-buffer pipeline includes the movement required to process a host-resident buffer. The different outcomes observed under these boundaries show that the computational benefit of GPU execution can be reduced or eliminated by additional processing operations.

The second constraint is the existing performance of the host CPU. Hardware-assisted AES-NI can provide substantial encryption throughput, and a GPU advantage must be established against the available host implementation rather than assumed from an unaccelerated software reference.

The third constraint is encryption-mode behavior. The T4 results demonstrate that similar absolute GPU throughput can produce different relative acceleration outcomes when the corresponding CPU modes have substantially different execution rates. The underlying cause of this host behavior was not isolated. Accordingly, the findings do not support attributing the observed difference to a particular processor feature, library version, or cloud mechanism.

The fourth constraint is workload size. GPU execution introduces fixed and variable costs that may be less favorable for smaller processing requests.

The experimental size grid also exhibits non-monotonic behavior, preventing the identification of a simple universal CPU-to-GPU crossover threshold.

These constraints suggest that a deployment-oriented execution policy would need to consider workload characteristics and data location rather than selecting an accelerator solely on the basis of peak kernel throughput. An adaptive selection mechanism could potentially assign suitable bulk workloads to the GPU while retaining CPU execution for other requests. Determining an effective policy would require workload-specific measurements and remains outside the demonstrated results.

### 5.5 Experimental Validity and Generalization

The observed advantages are supported by repeated measurements under the tested conditions, but several factors limit their generalization. Each configuration represents one host or cloud allocation with five study sessions. These sessions provide evidence of repeatability within the selected environment, not a statistical characterization of the wider population of comparable machines.

The configurations differ in processor models, GPU devices, operating systems, drivers, and supporting software. Their performance differences therefore cannot be attributed causally to a single hardware or software characteristic. The CPU reference is a selected AES-NI-enabled library path rather than a comprehensive evaluation of the fastest possible multithreaded CPU encryption service. Similarly, the CUDA kernel is a validated implementation, not a demonstrated performance limit of the GPUs.

The prepared-buffer pipeline excludes initial host staging, allocation, authentication, external input/output, and application-level orchestration. Its results must therefore not be interpreted as complete telecommunications-system throughput. The fixed ascending order of input sizes also limits causal interpretation of the observed non-monotonic performance behavior. In particular, the local 16 MiB reduction cannot be assigned to memory effects, GPU scheduling, processor clocks, or thermal conditions without additional evidence.

Finally, the use of five-session ranges and directional consistency does not establish formal statistical significance. The results identify observed repeatable conditions rather than guaranteed future performance.

These limitations define the boundary of the contribution: validated evidence of bulk AES processing improvements under measured conditions, not a universal claim about GPU cryptographic acceleration.

### 5.6 Future Research Directions

The most direct extension is the evaluation of GPU-assisted AES processing within a representative telecommunications workload, particularly a communication gateway or bulk data-protection service.

Such an investigation could determine whether the observed AES-CTR improvements remain beneficial when packet aggregation, cryptographic context management, and output delivery are included.

A second direction concerns authenticated encryption. A GPU implementation of AES-GCM, including GHASH, authentication-tag processing, and the required security state, would provide a more direct basis for evaluating GPU acceleration in IPsec-oriented systems.

A third direction is the investigation of workload-aware CPU–GPU scheduling. Such a system could evaluate whether processing volume, data residency, and latency requirements justify execution on the GPU or the host CPU.

A fourth direction involves transfer-aware processing techniques, including overlapping communication and computation, persistent device-resident data, and supported direct-data-transfer mechanisms. These approaches could change the practical cost of cryptographic offloading, but their performance cannot be predicted from the synchronous measurements alone.

Finally, additional independent systems and more densely sampled workloads would help establish the external validity of the observed acceleration conditions.

These extensions are not prerequisites for the present empirical contribution. Rather, they identify the necessary steps for translating measured bulk AES acceleration into deployable cryptographic services for telecommunications infrastructure.

## 6 Conclusion

This study demonstrates that GPU-assisted AES encryption can improve bulk cryptographic processing performance under specific operating conditions, even when the host processor already supports hardware-accelerated AES-NI. The contribution is the experimental identification of conditions under which this improvement remains observable after including synchronous host–device transfers, together with the practical constraints that determine its potential relevance to telecommunications systems.

The strongest evidence is obtained for AES-128 CTR encryption of 32 MiB buffers. The validated CUDA pipeline achieved a median paired acceleration of $1.354\times$ on the T600 configuration and $4.761\times$ on the T4 configuration, relative to their respective host AES-NI implementations. Both improvements were observed

in all five experimental sessions. The T600 result is particularly relevant because it establishes that a repeatable bulk-processing advantage is possible on the tested 4 GB laptop GPU, without relying on a datacenter-class accelerator.

However, the experiments also establish important boundaries. The GTX configuration did not provide a consistent transfer-inclusive improvement over AES-NI, while the T4 ECB pipeline remained slower than its corresponding CPU reference. Although device-resident GPU execution consistently outperformed AES-NI at 32 MiB, the advantage did not always survive the inclusion of data movement and synchronization. The results therefore identify workload-dependent acceleration opportunities rather than a universal GPU performance advantage.

For telecommunications infrastructure, these findings support further consideration of GPU-assisted bulk encryption, particularly in systems capable of processing large buffers or organizing independent cryptographic operations into suitable workloads. AES-CTR parallelism provides a relevant computational basis for such architectures, including potential acceleration of the encryption component within IPsec-oriented processing systems. Nevertheless, the experiments do not establish improvements in complete IPsec ESP, AES-GCM, or packet-processing pipelines, where authentication, protocol state, batching, and communication latency introduce additional requirements.

The validation-first experimental design, paired measurements, explicit timing boundaries, and preserved session-level evidence support the reproducibility and interpretation of the reported findings. The principal implication is that GPU-assisted AES should be investigated as a targeted processing strategy for suitable bulk cryptographic workloads, rather than as a direct replacement for CPU-based encryption. The findings provide experimentally grounded operating conditions and practical constraints that can inform the development of heterogeneous telecommunications architectures, while identifying authenticated encryption, packet aggregation, and workload-aware execution as areas requiring application-level evaluation.

## References


[1] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[3] Svetlin A. Manavski. Cuda compatible gpu as an efficient hardware accelerator for aes cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, 2007. doi: 10.1109/ICSPC.2007.4728256.

[4] Owen Harrison and John Waldron. Aes encryption and decryption on the gpu. In *GPU Gems 3*,

chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu.

[5] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of aes encryption on cuda gpu. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[6] Canhui Wang and Xiaowen Chu. Gpu accelerated aes algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[7] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[8] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of aes-ctr and aes-ecb bit-sliced implementation on gpus. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[9] Intel Corporation. Intel advanced encryption standard instructions (aes-ni), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x gpu vs. cpu myth: An evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[11] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[12] NVIDIA Corporation. Cuda c++ best practices guide, n.d. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[13] Mark Harris. How to overlap data transfers in cuda c/c++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[14] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[15] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[16] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, 2013. doi: 10.1145/2464157.2464160.

[17] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[18] PyCryptodome. Pycryptodome documentation, n.d. URL https://pycryptodome.readthedocs.io/. Accessed 2026.

[19] CuPy. cupy.rawkernel. https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html, n.d.. Accessed 2026.

[20] CuPy. Performance best practices. https://docs.cupy.dev/en/stable/user_guide/performance.html, n.d.. Accessed 2026.

[21] Russ Housley. Using advanced encryption standard (AES) counter mode with IPsec encapsulating security payload (ESP). RFC 3686, Internet Engineering Task Force, January 2004. URL https://www.rfc-editor.org/info/rfc3686.

[22] John Viega and David A. McGrew. The use of galois/counter mode (GCM) in IPsec encapsulating security payload (ESP). RFC 4106, Internet Engineering Task Force, June 2005. URL https://www.rfc-editor.org/info/rfc4106.